# LSP Telemetry System

*Software Design Specification*

Version 1.0
24/01/2026

Contributors:

Saran Saravanan

# Document History

**Revision History**

| Revision # | Revision Date | Description of Change | Author |
|---|---|---|---|
| 1 | 24/01/2026 | First Draft | Saran S |
| | | | |
| | | | |
| | | | |
| | | | |

1. Introduction
    1.1 Purpose
    1.2 Overview
2. System Architecture
    2.1 Architectural Design
    2.2 Decomposition Description
    2.3 Design Rationale
3. Web Application
    3.1 Implementation Design
    3.2 Rationale and Breakdown
4. Engine
    4.1 Implementation Design
    4.2 Rationale and Breakdown
5. Customize Item Explorer
    5.1 Implementation Design
    5.2 Rationale and Breakdown
6. Algorithms
    6.1 Sequence Diagram
    6.2 Rationale and Breakdown
7. Database
    7.1 Implementation Design
    7.2 Rationale and Breakdown

# 1. Introduction

## 1.1 Purpose

This document will have all the implementation designs and rationales for why the system was designed in the manner it was designed. This document will help new developers spin up on the project and remind current developers why certain design decisions were made.

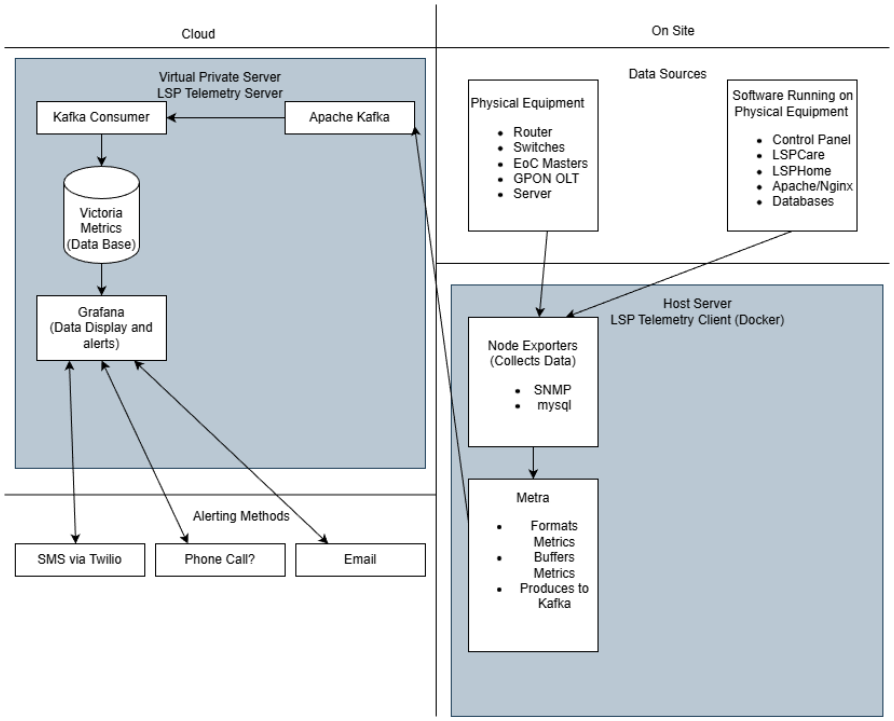
## 1.2 Overview

The system architecture section of this document will contain the bird's eye view of the architecture for the project. This was a simple architecture that was done up front to guide the design for all features of the application. This follows our methodology's core concepts for simplicity and to not have big design up front.

The other sections in this document will be different components or modules that are used in our project. This document will act as the one stop shop for design rationale and any potential future work for these components.

# 2. System Architecture

## 2.1 Architectural Design




## 2.2 Decomposition Description

The two main pieces of our architecture, the telemetry client and server. The split was chosen so the software is completely modular, the telemetry server is hosted on our cloud VPS and the client is run on the onsite server.

**Telemetry Client (On-Site)**

The client runs on the on-site server and is responsible for collecting telemetry data from our local infrastructure. We will utilize Prometheus exporters to collect data from our multiple sources using multiple protocols (SNMP, mysql, etc). This data will be scraped at set intervals by METPA and will be sent to a Kafka topic. If Kafka to down, due to network or other issues, METPA will collect and store the data and will continue to retry until successful.

**Telemetry Server (Cloud VPS)**

The telemetry server is hosted on a cloud-based virtual private server (VPS) and is responsible for ingestion, processing, storage, and visualization of telemetry data.

Apache Kafka serves as the central streaming platform, receiving telemetry events from multiple clients. Kafka consumers process the incoming data and store it in VictoriaMetrics, which acts as the time-series database. Grafana is used to visualize metrics and manage alerting rules.

Alerting can be sent via multiple channels, including SMS (via Twilio), email or phone calls.

## 2.3 Design Rationale

A streaming-based architecture was chosen to provide flexibility, scalability, and resilience for the monitoring system.

One key advantage of this approach is horizontal scalability. If a customer requires complete telemetry isolation, a new telemetry server instance can be deployed in the cloud, and the on-site client can be redirected simply by updating DNS records. This allows customer environments to be logically or physically separated without changes to the client software.

Kafka plays a critical role by acting as a **buffer and decoupling layer** between telemetry producers (clients) and consumers (processing services). This ensures:

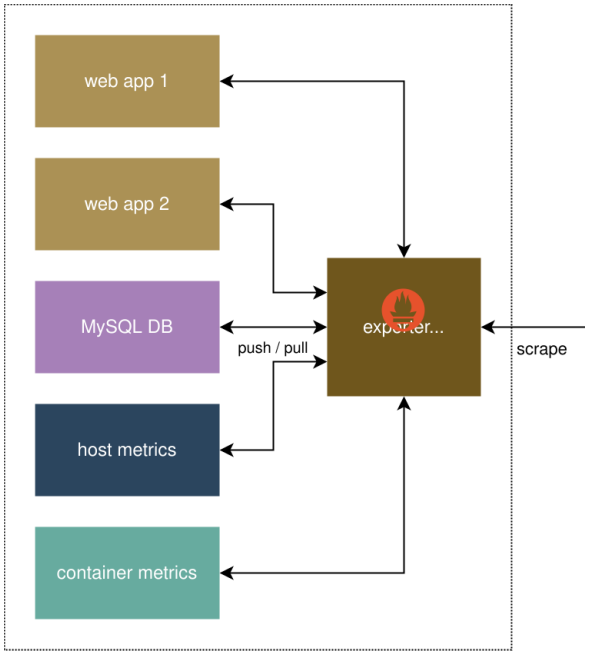
- Temporary spikes in telemetry data do not overwhelm downstream systems
- Consumers can be scaled independently
- Data can be replayed or reprocessed if needed

## 3. Requirements

WRITE REQUIREMENTS HERE

# 4. Prometheus Exporters

## 4.1 Implementation Design




## 4.2 Rationale and Breakdown

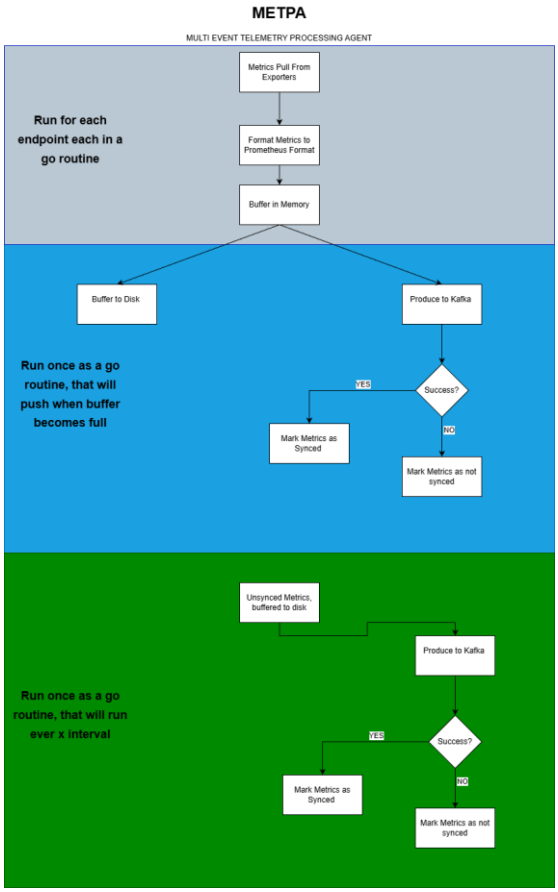
Prometheus Exporters were selected as the foundation for metric collection for several key reasons:

- **Standardized Metric Format**
  Exporters follow a consistent and well-defined data structure, ensuring compatibility across a wide range of systems and simplifying integration efforts.
- **Availability of Ready-Made Dashboards**
  A large ecosystem of pre-built dashboards exists for the Prometheus metrics format, reducing development time and enabling faster visualization and analysis.
- **Strong Open-Source Community Support**
  Prometheus and its exporters benefit from a mature, active open-source community, providing extensive documentation, community-driven improvements, and ongoing maintenance.
- **Future Scalability and Extensibility**
  The exporter model allows easy expansion if monitoring requirements grow or if additional types of hardware, software, or services need to be incorporated.

# 5. MET-PA — Multi-Event Telemetry Processing Agent

Commented [SS1]: Can change the name to something else.

## 5.1 Implementation Design




## 5.2 Rationale and Breakdown

A custom tool was built due to the specific requirements and the environment could not be fulfilled by existing tools such as Vector or Fluent Bit. The main reasons were:

- No support for compression on data store to disk.
  This would be a problem for sites where they can be offline for weeks (hospitals with 4G modem). Metrics can accumulate to be hundreds of gigabytes.

Commented [SS2]: I believe so, but haven't looked too far into this

- Nonstandard metric format when pulling metrics from Prometheus exporters. No labels included such as job or instance which is required for dashboards in Grafana. Additionally, poor support to add custom labels like site identification (site_id)
- Business requirements state that data is be retained (x internal) even after being pushed to Kafka. Which is not possible without adding another tool

Hence, it was decided that a custom tool written in GO was to be built. GO was chosen for a couple of reasons

- Easy of development while maintaining high performance
- Compiled binary, no separate runtime required
- Native multi-threading support with go routines (important for our use case)

The tool was designed to be like fluent bit and vector, while being more focused on our use case.

**Commented [SS3]:** Fluent Bit can add labels but it adds them only on specific outputs and that doesn't include Kafka. Might be able to get around this with a formatter like lua script which is supported by fluent bit.

Vector has the same problem as well.

Again needs to be looked at in more detail.

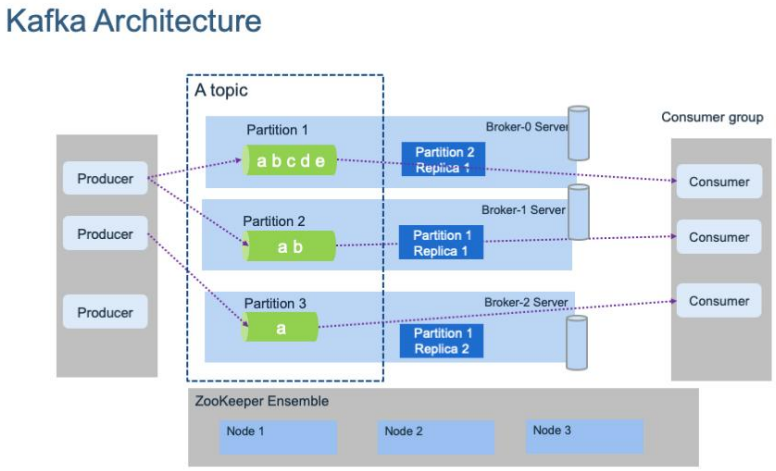
**Commented [SS4]:** This is a custom requirement which most tools were not designed for.

Solution would be to write to prometheues using remote write and set retention to 7 days.

**Commented [SS5]:** Sort of started writing the tool but needs to be looked at if we can get away with an existing tool.

# 6. Kafka

## 6.1 Implementation Design




## 6.2 Rationale and Breakdown

Apache Kafka provides a reliable, scalable way to stream telemetry data from the on-site agents. It was chosen specifically to keep the producers (METPA) and consumers loosen coupled.

- Producers send telemetry events into Kafka topics.
- Topics/Partitions organise data and enable parallel processing.
- Cluster replicates data across brokers for reliability.
- Consumers read streams for analytics, monitoring, and storage.
- Decouples producers/consumers for flexible system design.

# 7. Victoria Metrics (Database)

## 7.1 Implementation Design




## 7.2 Rationale and Breakdown

We knew the format of information that we were getting from the Wegmans team but we wanted to normalize it and have our own representation to suit the needs of our application. We did this to make data retrieval easier and to logically assist anyone who needed to look at the data in our database.

# 8. Grafana

## 8.1 Implementation Design



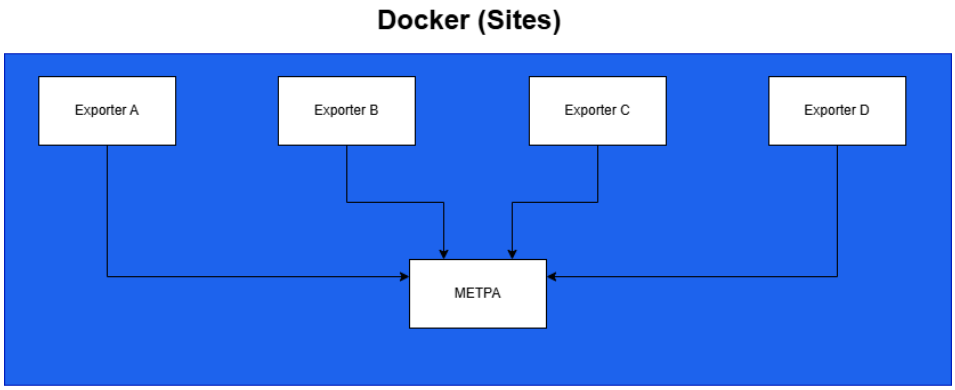

## 7.2 Rationale and Breakdown

Grafana was chosen for the frontend UI to display our data for the following reasons.

- Open source, widely adopted, and backed by strong community support.
- Native integration with Prometheus for seamless metric visualisation.
- Flexible and highly customisable dashboards (variables, panels, templates).
- Supports many data sources, enabling unified observability.
- Large plugin ecosystem for extended functionality.
- Built-in alerting system with multi-channel notification options.
- Easy to deploy, configure, and maintain in telemetry environments.
- Scales well for high-volume, real-time metric workloads.
- Intuitive UI that accelerates monitoring and operational insights.

# 9. Docker

## 9.1 Implementation Design




## 7.2 Rationale and Breakdown

Docker will be utilized to run the exporters and METPA on the site servers. This was chosen as it allows the telemetry client system to be easily deployed and again follows the principle of modularity. For our cloud VPS, it has been opted to install the software directly on the host. This was chosen for maximum performance, especially when aggregating data from multiple sites (60+), where we could see 1M/min of metrics.